NUS SCHOOL OF COMPUTING · CS5224 CLOUD COMPUTING · TEAM 18

SeatRelay

One waitlist for every session of a workshop.

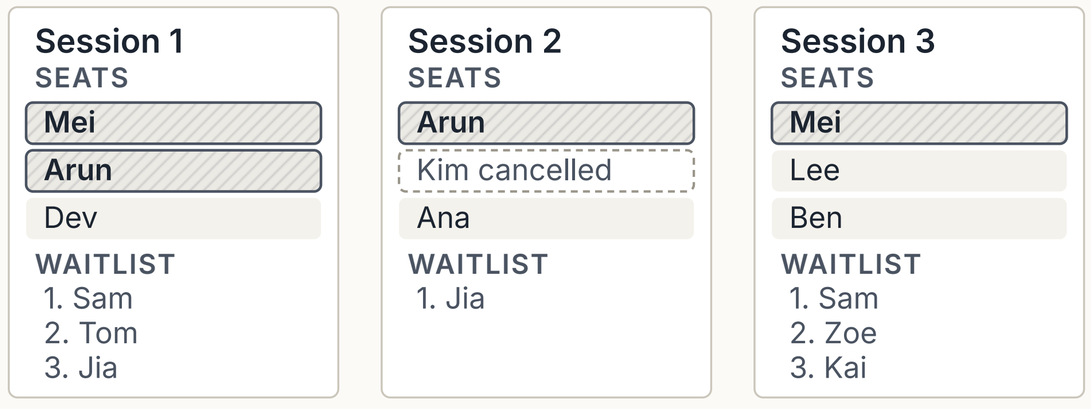
Register once and rank the sessions you can attend. A freed seat is held for the next person in line who can attend that session, not left empty or waiting for an organiser's e-mail.

Li Zongze · Chen Yang · Lin Jie · Zhu Gengyu · Liang Zihan · Zhang Tianyi

Live app on AWS
dev stage · d2ni5e1z674t04.cloudfront.net

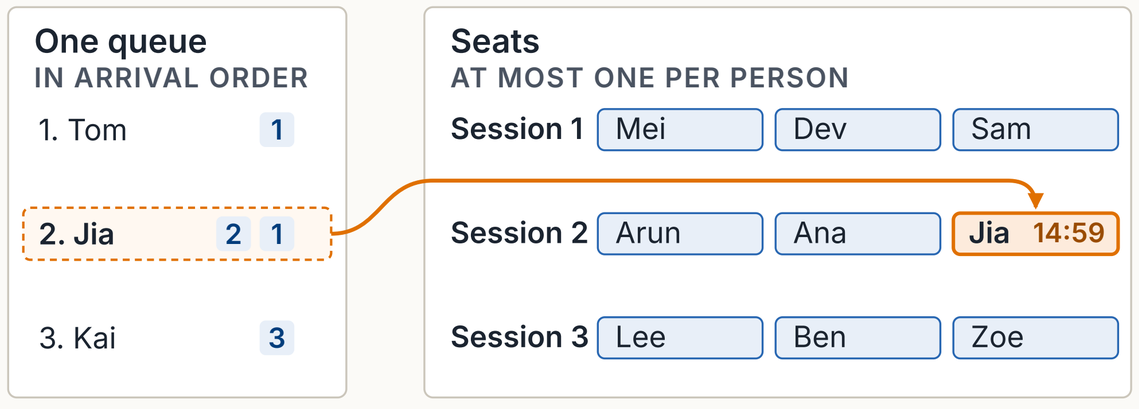
Why one form per session ties up seats

Today: a separate sign-up form for each session



Mei and Arun hold two seats each; others miss out.
Kim cancelled: the seat waits for an organiser's e-mail.

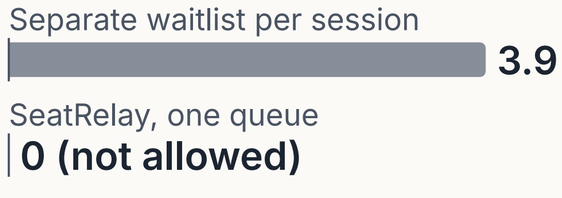
SeatRelay: one queue, at most one seat each



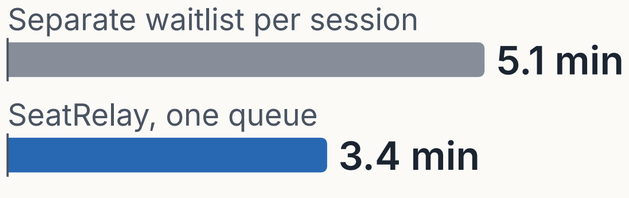
Kim cancelled: Tom wants only Session 1, so Jia gets the hold.
Chips: the sessions each person accepts, best first.
No organiser step: the hand-over is part of the same update.

Simulated on 1,000 random workshops: the same arrivals replayed through both designs

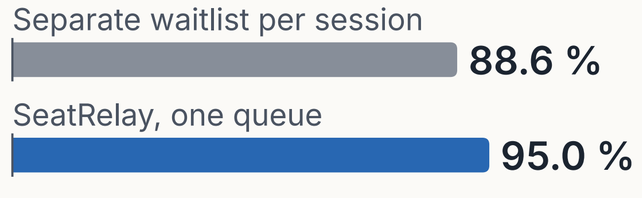
Second seats held at the end



Minutes to refill a freed seat



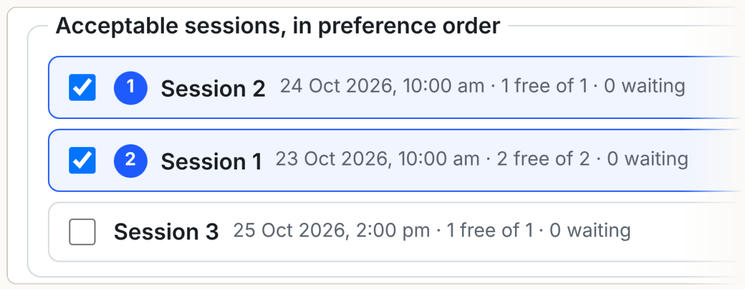
Seats given to different people



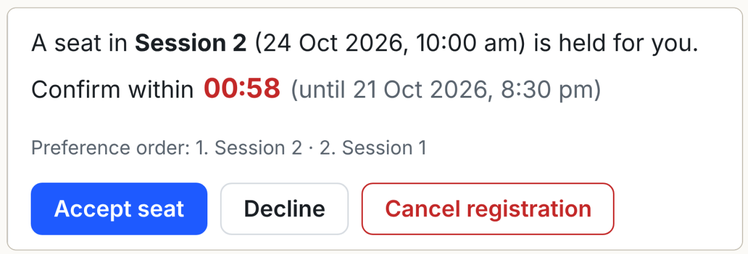
A simulation, not field data: our allocation engine with assumed behaviour; means per workshop, refill time (incl. people's replies) as the median workshop (means 15.7 vs 4.3 min). The baseline's waitlists are automatic too, so the gap comes only from people holding several seats.

How it works

1 Tick every session you could attend and rank them.



2 Wait in one queue. When a seat you can take frees up and you are first, it is held for you (15 min by default).



Staged run with test accounts and 1-minute holds.

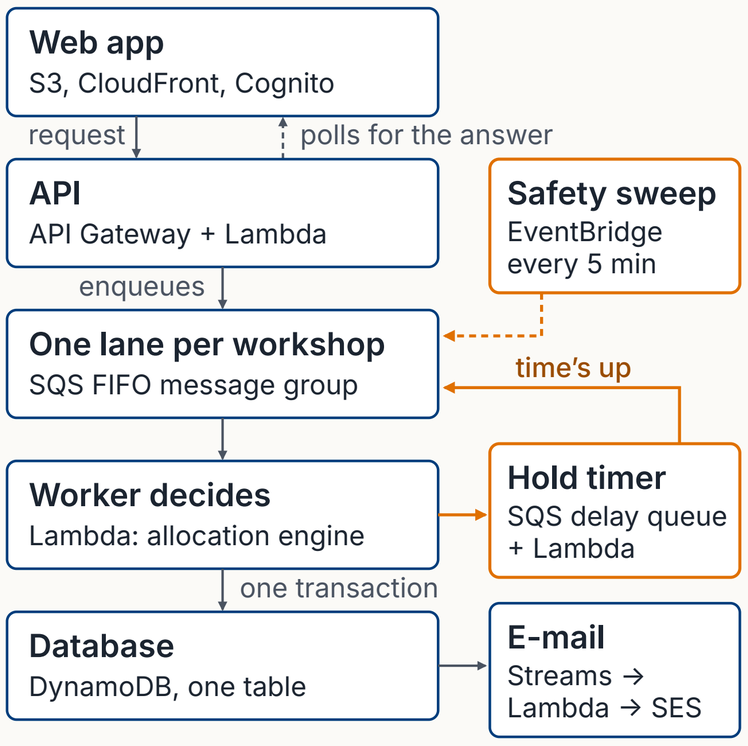
3 Accept to keep it. Decline or let it lapse: you lose only that session, and the seat moves on at once.

4 Organisers see every hand-over and its delay.

Released	Session	Cause	Reallocated	Latency
8:30:31 pm	Session 2	offer expired	8:30:31 pm	189 ms
8:29:31 pm	Session 2	confirmed seat cancelled	8:29:31 pm	0 ms

Top row: an offer expired; the seat was re-held 189 ms later.

Serverless on AWS



- Each workshop's requests are decided in arrival order; a version check catches any overlap.
- Seats, queue, offer and notice change in one transaction.
- Each hold has its own timer, and a late timer cannot touch a newer offer.

Cost model, list prices: US$13.30 per 1,000 registrations at 1k a month, US$5.91 at 100k. An own server is cheaper above about 57,000 a month if fully costed, or about 3,900 if one is already running.

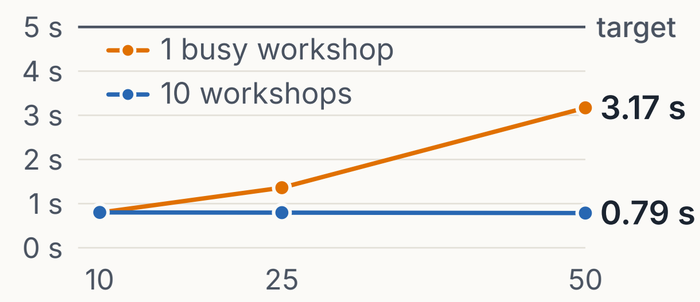
Measured on AWS

8/8 scripted experiments from our proposal (Table 3) pass on AWS

A first run hit the new account's Lambda limit of 10 and lost requests but broke no rule; we re-ran everything at 1,000.

0	overbooked sessions or people with two seats (672 checks)
50	accounts raced for 5 seats; every invariant held (39 checks)
0.10 s	from a hold expiring to the next person's hold (p95; target 120 s)
3/3	forced worker crashes: each request applied once on the queue's 2-min retry (target 5 min)

Time to the final answer (p95)



simulated users (k6), each registering non-stop

k6; median of 3 runs per point; 0.00 % errors. One workshop is decided one request at a time, so it slows first.

Next: a user study with twelve students and a live two-session pilot with real participants.

Code, data and every measurement: github.com/gengyuzccc-cell/Team18_CS5224_SeatRelay_Project_AY2627Sem1 (results/). Measured on our AWS deployment (ap-southeast-1) on 17 Oct 2026, UTC; screens from a staged run with test accounts on 21 Oct 2026.

AI use: Claude Code, an AI coding assistant, generated the code, infrastructure templates, tests and experiment scripts, ran the deployments and measurements with our credentials, and drafted this poster's text and SVG figures. The scope, allocation rules, constraints and acceptance targets are the team's (Final report, AI declaration). No image-generation model was used: the charts are plotted from results/ and the screens are screenshots of the app.

code & data